GVFG 客戶 API Reference

目錄

1. GVFG API 結構與常數
   1.1 公開常數
   1.2 gvfg_status_t
   1.3 gvfg_pixel_format_t
   1.4 gvfg_channel_t
   1.5 gvfg_video_interface_t
   1.6 gvfg_sdi_mode_t
   1.7 gvfg_sdi_resolution_t
   1.8 gvfg_sdi_fps_t
   1.9 gvfg_device_info_t
   1.10 gvfg_sdi_info_t
   1.11 gvfg_signal_status_t
   1.12 gvfg_runtime_info_t
   1.13 gvfg_frame_t
   1.14 gvfg_audio_format_t
   1.15 gvfg_audio_frame_t
   1.16 gvfg_gpu_output_format_t
   1.17 gvfg_gpu_output_buffer_t
   1.18 gvfg_event_type_t
   1.19 gvfg_handle
   1.20 gvfg_channel_osd_config_t
2. GVFG API 函式
   2.1 gvfg_enumerate_devices
   2.2 gvfg_create
   2.3 gvfg_destroy
   2.4 gvfg_open_channel
   2.5 gvfg_get_channel_sdi_info
   2.6 Zero-copy mode selection
   2.7 gvfg_set_channel_video_format
   2.8 Audio capture selection and frame ownership
   2.9 gvfg_start_channel
   2.10 gvfg_read_channel_frame
   2.11 gvfg_release_channel_frame
   2.12 gvfg_gpu_convert_to_buffer
   2.13 gvfg_gpu_convert_to_bgra8
   2.14 gvfg_gpu_convert_to_rgb10a2
   2.15 gvfg_gpu_convert_to_nv12
   2.16 gvfg_poll_channel_event
   2.17 gvfg_stop
   2.18 gvfg_stop_channel
   2.19 gvfg_get_channel_signal_status
   2.20 gvfg_get_channel_runtime_info
   2.21 gvfg_get_version
   2.22 gvfg_pixel_format_name
   2.23 gvfg_strerror
   2.24 gvfg_get_channel_last_sdk_error_detail
   2.25 gvfg_measure_osd_text
   2.26 gvfg_set_channel_osd / gvfg_clear_channel_osd
3. Playback 使用的影像型別
   3.1 Preview 型別
4. 統一 Playback API
   4.1 gvfg_playback_config_t
   4.2 gvfg_playback_video_frame_t
   4.3 gvfg_playback_submit_audio() 的輸入
   4.4 狀態碼
   4.5 gvfg_playback_stats_t
   4.6 控制 API 的細節

SDK 2.1.1。Capture 與 Playback 使用各自的 API 與狀態碼。

- 擷取型別與常數
- 擷取函式
- 播放影像型別
- 播放 API、config、stats 與限制

1. GVFG API 結構與常數

說明 `gvfg_capture.h` 的所有公開常數、型別、結構。除非特別註明， 所有輸出結構都建議先用 `{0}` 初始化。

1.1 公開常數

常數	值	說明
`GVFG_MAX_DEVICES`	16	一次最多列舉的裝置數
`GVFG_TIMEOUT_INFINITE`	`UINT32_MAX`	無限等待； `timeout_ms == 0` 則完全不等待
`GVFG_OSD_MAX_TEXT_CHARACTERS`	32	OSD 可列印 ASCII 字元數上限

1.2 `gvfg_status_t`

`gvfg_status_t` 是 32-bit 整數。`0` 表示成功，失敗時回傳負數的 `GVFG_E*` 狀態。

成員	值	說明
`GVFG_OK`	0	成功
`GVFG_EINVAL`	-1	NULL pointer、無效 channel、buffer 大小或 token 等參數錯誤
`GVFG_ENODEV`	-2	找不到裝置或裝置無法開啟
`GVFG_ESTATE`	-3	呼叫順序或目前 session 狀態不允許此操作
`GVFG_EIO`	-4	SDK I/O、GPU 或其他處理失敗
`GVFG_ENOTSUP`	-5	不支援指定功能或格式
`GVFG_ETIMEOUT`	-6	在期限內等不到 frame、event 或操作完成
`GVFG_EBUSY`	-7	裝置或 capture resource 正在使用中

統一以 `status != GVFG_OK` 判斷失敗，不可使用 Windows `FAILED()` / `SUCCEEDED()`。 `gvfg_strerror()` 回傳對應的公開英文錯誤說明。

1.3 `gvfg_pixel_format_t`

成員	值	說明
`GVFG_PIXFMT_UNKNOWN`	0	未知或尚無有效訊號格式
`GVFG_PIXFMT_YUY2`	1	8-bit packed YUV 4:2:2；byte order Y0 U0 Y1 V0，每 pixel 2 bytes
`GVFG_PIXFMT_Y210`	2	10-bit packed YUV 4:2:2；目前每 pixel 4 bytes

1.4 `gvfg_channel_t`

成員	值	說明
`GVFG_CHANNEL_0`	0	裝置輸入通道 0
`GVFG_CHANNEL_1`	1	裝置輸入通道 1

1.5 `gvfg_video_interface_t`

成員	值	說明
`GVFG_INPUT_INTERFACE_SDI`	0	SDI input
`GVFG_INPUT_INTERFACE_HDMI`	1	HDMI input

1.6 `gvfg_sdi_mode_t`

成員	值	說明
`GVFG_SDI_INPUT_MODE_HD`	0	HD-SDI input
`GVFG_SDI_INPUT_MODE_SD`	1	SD-SDI input
`GVFG_SDI_INPUT_MODE_3G`	2	3G-SDI input

1.7 `gvfg_sdi_resolution_t`

成員	值	說明
`GVFG_SDI_INPUT_RESOLUTION_SMPTE_ST_274_1920X1080`	`0x0`	SMPTE ST 274 · 1920 x 1080
`GVFG_SDI_INPUT_RESOLUTION_SMPTE_ST_296_1280X720`	`0x1`	SMPTE ST 296 · 1280 x 720
`GVFG_SDI_INPUT_RESOLUTION_SMPTE_2048_2048X1080`	`0x2`	SMPTE 2048 · 2048 x 1080
`GVFG_SDI_INPUT_RESOLUTION_SMPTE_295_1920X1080`	`0x3`	SMPTE 295 · 1920 x 1080
`GVFG_SDI_INPUT_RESOLUTION_NTSC_720X486`	`0x8`	NTSC · 720 x 486
`GVFG_SDI_INPUT_RESOLUTION_PAL_720X576`	`0x9`	PAL · 720 x 576
`GVFG_SDI_INPUT_RESOLUTION_UNKNOWN`	`0xF`	未知或不支援的 resolution

1.8 `gvfg_sdi_fps_t`

成員	值	說明
`GVFG_SDI_INPUT_FPS_NONE`	`0x0`	無可用 frame rate
`GVFG_SDI_INPUT_FPS_23_98`	`0x2`	23.98 fps
`GVFG_SDI_INPUT_FPS_24`	`0x3`	24 fps
`GVFG_SDI_INPUT_FPS_47_95`	`0x4`	47.95 fps
`GVFG_SDI_INPUT_FPS_25`	`0x5`	25 fps

成員	值	說明
`GVFG_SDI_INPUT_FPS_29_97`	`0x6`	29.97 fps
`GVFG_SDI_INPUT_FPS_30`	`0x7`	30 fps
`GVFG_SDI_INPUT_FPS_48`	`0x8`	48 fps
`GVFG_SDI_INPUT_FPS_50`	`0x9`	50 fps
`GVFG_SDI_INPUT_FPS_59_94`	`0xA`	59.94 fps
`GVFG_SDI_INPUT_FPS_60`	`0xB`	60 fps

1.9 `gvfg_device_info_t`

由 `gvfg_enumerate_devices()` 填入。

欄位	型別	說明
`name`	`char[128]`	供 UI / log 顯示的 UTF-8、null-terminated 裝置名稱

1.10 `gvfg_sdi_info_t`

由 `gvfg_get_channel_sdi_info()` 填入。數值欄位可供程式判斷；對應的 `*_name` 欄位是可直接用於 UI 或 log 的 null-terminated 英文字串。

欄位	型別	說明
`connected`	`int`	非 0 表示 SDI 輸入訊號已 lock
`mode`	`gvfg_sdi_mode_t`	偵測到的 SDI link mode
`resolution`	`gvfg_sdi_resolution_t`	偵測到的 SDI resolution code
`fps`	`gvfg_sdi_fps_t`	偵測到的 SDI frame-rate code
`progressive`	`int`	非 0 為 progressive；0 為 interlaced
`level_b`	`int`	非 0 表示 3G-SDI Level B
`st352_payload`	`uint32_t`	SMPTE ST 352 payload identifier
`error_count`	`uint32_t`	裝置回報的 signal error count
`signal_lock_name[32]`	`char[32]`	Signal lock 說明
`mode_name[16]`	`char[16]`	SDI mode 名稱
`resolution_name[64]`	`char[64]`	Resolution 名稱
`fps_name[24]`	`char[24]`	Frame-rate 名稱
`scan_name[16]`	`char[16]`	Scan mode 名稱
`st352_format_name[64]`	`char[64]`	ST 352 format 名稱
`st352_fps_name[24]`	`char[24]`	ST 352 frame-rate 名稱
`st352_chroma_name[36]`	`char[36]`	ST 352 chroma 名稱

欄位	型別	說明
`st352_bit_depth_name[24]`	`char[24]`	ST 352 bit-depth 名稱

1.11 `gvfg_signal_status_t`

由 `gvfg_get_channel_signal_status()` 填入。

欄位	型別	說明
`connected`	`int`	非 0 表示查詢的 channel 目前有有效輸入訊號
`channel`	`int`	本次查詢對應的 `gvfg_channel_t` 值
`width`	`int`	連線時的輸入寬度；未連線時為 0
`height`	`int`	連線時的輸入高度；未連線時為 0
`pixel_format`	`int`	實際 frame payload 的 `gvfg_pixel_format_t` 值
`bit_depth`	`int`	從 payload 格式取得的每色彩 channel bit depth
`video_interface`	`int`	`gvfg_video_interface_t`，為 SDI 或 HDMI

1.12 `gvfg_runtime_info_t`

由 `gvfg_get_channel_runtime_info()` 填入。Channel 從 stopped 成功進入新的 running session 時重設統計；對已 running 的 channel 再次呼叫 Start 不會重設。

欄位	型別	說明
`capture_fps`	`double`	依 `gvfg_read_channel_frame()` 成功交付時間估算的 FPS；尚無足夠 frame 時為 0
`delivered_frames`	`uint64_t`	此次 running session 中成功交付給 caller 的 frame 數

1.13 `gvfg_frame_t`

由 `gvfg_read_channel_frame()` 填入，也是稍後傳給 `gvfg_release_channel_frame()` 的 frame descriptor。

欄位	型別	說明
`data`	`const void *`	借用的 frame buffer；release 或 stop 後失效
`data_size`	`uint64_t`	此 frame payload 的總 byte 數
`width`	`int`	frame 寬度，單位 pixel
`height`	`int`	frame 高度，單位 pixel
`row_stride_bytes`	`int`	相鄰兩列起點間距，單位 byte；處理每列時必須使用此值
`pixel_format`	`int`	`gvfg_pixel_format_t` 值
`bit_depth`	`int`	原生 frame 每色彩 channel 的 bit depth
`frame_id`	`uint64_t`	SDK 成功交付序號；每次 channel Start 從 1 重新開始
`timestamp_ns`	`uint64_t`	SDK 交付時間；與 audio 共用 monotonic clock，單位 ns

Caller 應把 read 取得的 descriptor 傳回 release；SDK 以 `data + frame_id` 確認目前 held frame。

1.14 `gvfg_audio_format_t`

由 `gvfg_get_channel_audio_format()` 填入，只包含 Application 建立播放或錄音 格式所需的 PCM metadata。

欄位	型別	說明
`sample_rate`	`uint32_t`	每秒 sample 數
`channels`	`uint32_t`	interleaved PCM channel 數
`bits_per_sample`	`uint32_t`	每個 PCM sample 的 bit 數

每個 audio frame 的有效資料長度由 `gvfg_audio_frame_t.data_size` 提供。

1.15 `gvfg_audio_frame_t`

由 `gvfg_read_channel_audio_frame()` 填入，也是稍後傳給 `gvfg_release_channel_audio_frame()` 的 audio frame descriptor。

欄位	型別	說明
`data`	`const void *`	借用的 interleaved PCM；release 或 stop 後失效
`data_size`	`uint64_t`	此 frame 的有效 PCM byte 數
`sample_rate`	`uint32_t`	每秒 sample 數
`channels`	`uint32_t`	interleaved PCM channel 數
`bits_per_sample`	`uint32_t`	每個 PCM sample 的 bit 數
`frame_id`	`uint64_t`	SDK 成功交付序號；每次 channel Start 從 1 重新開始
`timestamp_ns`	`uint64_t`	SDK 交付時間；與 video 共用 monotonic clock，單位 ns

與 video 相同，每個 channel 同時只能持有一個 audio frame；release 以 `data + frame_id` 確認目前 held frame。

Video/audio 的 `timestamp_ns` 可在同一 process/session 內直接比較。它代表 SDK 交付時間，不代表訊號來源端產生 frame 的時間。

1.16 `gvfg_gpu_output_format_t`

成員	值	說明
`GVFG_GPU_OUTPUT_BGRA8`	1	DXGI `B8G8R8A8_UNORM` byte layout，alpha 為 255
`GVFG_GPU_OUTPUT_RGB10A2`	2	DXGI `R10G10B10A2_UNORM` packed `uint32_t`，alpha 為 3
`GVFG_GPU_OUTPUT_NV12`	3	8-bit BT.709 limited-range 4:2:0；Y plane 後接 interleaved UV

1.17 `gvfg_gpu_output_buffer_t`

由 caller 建立並傳給 `gvfg_gpu_convert_to_buffer()`。

欄位	型別	說明
`data`	`void *`	Caller-owned destination buffer，不可為 NULL
`data_size`	`uint64_t`	Destination buffer 可用 byte 數
`row_bytes`	`int`	Destination stride；NV12 的 Y/UV plane 共用此 stride
`pixel_format`	`int`	要求的 `gvfg_gpu_output_format_t` 值

例如，要建立一個與輸入 frame 同尺寸的 BGRA8 destination：

```
const int row_bytes = source.width * 4;
const uint64_t buffer_size = (uint64_t)row_bytes * (uint64_t)source.height;
void *pixels = malloc((size_t)buffer_size);

gvfg_gpu_output_buffer_t output = {0};
output.data = pixels;
output.data_size = buffer_size;
output.row_bytes = row_bytes;
output.pixel_format = GVFG_GPU_OUTPUT_BGRA8;

gvfg_status_t status = gvfg_gpu_convert_to_buffer(&source, &output);
/* 使用 pixels 後由 caller 呼叫 free(pixels)。 */
```

Caller 必須確認尺寸乘法沒有 overflow，並在 conversion 返回前保持 destination 有效。

1.18 `gvfg_event_type_t`

成員	值	說明
`GVFG_EVENT_UNKNOWN`	0	未知事件；正常流程不應依賴此值
`GVFG_EVENT_VIDEO_FORMAT_CHANGED`	1	輸入 video format 已變更
`GVFG_EVENT_VIDEO_INPUT_PLUGIN`	2	Video input 已連接
`GVFG_EVENT_VIDEO_INPUT_UNPLUG`	3	Video input 已中斷

SDK 會建立並管理完整的 channel event 集合；使用者直接透過 `gvfg_poll_channel_event()` 接收。

1.19 `gvfg_handle`

```
typedef struct gvfg_handle_t *gvfg_handle;
```

只能由 `gvfg_create()` 建立、由 `gvfg_destroy()` 銷毀；caller 不可取得、配置或修改其內部內容。

1.20 `gvfg_channel_osd_config_t`

由 caller 建立並傳給 `gvfg_set_channel_osd()`。SDK 在函式返回前複製文字及設定， caller 不必持續保留 `text` pointer。

欄位	型別	說明
`text`	`const char *`	可列印 ASCII（`0x20`–`0x7e`），不可為 NULL
`text_length`	`uint32_t`	1–32 個字元；不包含結尾 NUL
`x`、`y`	`int32_t`	完整量測文字框的左上角，單位 frame pixel；會限制在可完整顯示範圍
`font_size_px`	`uint32_t`	非零字號；SDK 沒有 viewer 的 512 px UI 上限
`color_rgb`	`uint32_t`	`0x00RRGGBB`；忽略最高 8 bits

使用 Arial-preferred normal 字型，固定單行不換行。文字框包含排版空白、glyph overhang 及四邊各 1 px 安全邊；用 `gvfg_measure_osd_text()` 取得與繪製相同的保守框大小。

2. GVFG API 函式

說明 `gvfg_capture.h` 的所有公開函式。

函式參數方向以 `[in]`、`[out]` 與 `[in, out]` 標示：`[in]` 由 caller 提供，`[out]` 由函式寫入，`[in, out]` 則同時包含 caller 必須先設定的輸入欄位與函式寫回的結果。

2.1 `gvfg_enumerate_devices`

```
int gvfg_enumerate_devices(/* [out] */ gvfg_device_info_t *out_devices,
                           /* [in] */ int max_devices);
```

參數：

- `out_devices`：接收裝置資料的陣列；只查數量時傳 `NULL`。
- `max_devices`：陣列可容納的項目數；超過 `GVFG_MAX_DEVICES` 會被截斷。當 `out_devices == NULL` 時應傳 0。

回傳值：有提供 output array 時為實際寫入數量；只查數量時為可用裝置數；沒有裝置 時為 0。此函式回傳數量，不回傳 `gvfg_status_t`。

2.2 `gvfg_create`

```
gvfg_status_t gvfg_create(/* [out] */ gvfg_handle *out_handle);
```

- `out_handle`：接收新 handle，不可為 NULL。
- 成功：`GVFG_OK`，handle 處於 closed 狀態。
- 失敗：`GVFG_EINVAL`（output pointer 為 NULL）。

2.3 `gvfg_destroy`

```
gvfg_status_t gvfg_destroy(/* [in] */ gvfg_handle handle);
```

- `handle`：`gvfg_create()` 回傳的 handle；可為 NULL。
- 回傳：`GVFG_OK`。若仍在 running，SDK 會先停止。返回後 handle 不得再使用。

2.4 `gvfg_open_channel`

```
gvfg_status_t gvfg_open_channel(/* [in] */ gvfg_handle handle,
                                /* [in] */ int device_index,
                                /* [in] */ int channel_index);
```

- `handle`：已建立的 session handle。
- `device_index`：`gvfg_enumerate_devices()` 結果中的 zero-based 陣列位置。
- `channel_index`：`GVFG_CHANNEL_0` 或 `GVFG_CHANNEL_1`。
- 可能回傳：`GVFG_OK`、`GVFG_EINVAL`、`GVFG_ENODEV`、`GVFG_ESTATE`、`GVFG_EBUSY`、`GVFG_EIO`。

對同一 handle 可分別 open CH0、CH1。已開啟任一 channel 後，不可使用同一 handle 切換至其他 device index。

2.5 `gvfg_get_channel_sdi_info`

```
gvfg_status_t gvfg_get_channel_sdi_info(
    /* [in] */ gvfg_handle handle,
    /* [in] */ int channel_index,
    /* [out] */ gvfg_sdi_info_t *out_info);
```

- `handle`：已 open 指定 channel 的 session。
- `channel_index`：要查詢的 `GVFG_CHANNEL_0` 或 `GVFG_CHANNEL_1`。
- `out_info`：接收 SDI 詳細資訊，不可為 NULL。
- `GVFG_OK`：查詢成功。
- `GVFG_EINVAL`：handle 或 output pointer 為 NULL，或 channel index 無效。
- `GVFG_ESTATE`：指定 channel 尚未 open。
- 裝置查詢失敗時可能回傳 `GVFG_EIO`。

2.6 Zero-copy mode selection

```
gvfg_status_t gvfg_set_channel_zero_copy_enabled(/* [in] */ gvfg_handle handle,
                                                 /* [in] */ int channel,
                                                 /* [in] */ int enabled);
gvfg_status_t gvfg_get_channel_zero_copy_enabled(/* [in] */ gvfg_handle handle,
                                                 /* [in] */ int channel,
                                                 /* [out] */ int *out_enabled);
```

- `set` 只能在 `gvfg_create()` 後、指定 channel open 前呼叫。
- `enabled` 只接受 0（copy）或 1（zero-copy）；每個 channel 預設為 0。
- 指定 channel 已 open 時呼叫 `set` 會回傳 `GVFG_ESTATE`；不影響另一條 channel。
- `get` 可查詢指定 channel 的模式；`out_enabled` 不可為 NULL。
- Zero-copy mode 的 `frame.data` 是借用的 pointer；仍必須以相同 descriptor 呼叫 `gvfg_release_channel_frame()`，且每個 channel 同時最多持有一張 frame。
- 此設定控制原生 frame 的擷取方式。OSD 等 SDK 處理可能交付 SDK-owned output buffer；caller 的 read / release 流程相同。

2.7 `gvfg_set_channel_video_format`

```
gvfg_status_t gvfg_set_channel_video_format(/* [in] */ gvfg_handle handle,
                                            /* [in] */ int channel_index,
                                            /* [in] */ gvfg_pixel_format_t format);
```

- 支援 `GVFG_PIXFMT_YUY2` 與 `GVFG_PIXFMT_Y210`。
- Channel 必須已 open，且 capture 必須尚未 start 或已 stop。
- 同一 handle 同時只能有一個 channel 選擇 Y210；衝突的要求回傳 `GVFG_EBUSY`。
- YUY2 對應 8-bit color depth，Y210 對應 10-bit。

2.8 Audio capture selection and frame ownership

```
gvfg_status_t gvfg_set_channel_audio_enabled(/* [in] */ gvfg_handle handle,
                                             /* [in] */ int channel_index,
                                             /* [in] */ int enabled);
gvfg_status_t gvfg_get_channel_audio_format(/* [in] */ gvfg_handle handle,
                                            /* [in] */ int channel_index,
                                            /* [out] */ gvfg_audio_format_t *out_format);
gvfg_status_t gvfg_read_channel_audio_frame(/* [in] */ gvfg_handle handle,
                                            /* [in] */ int channel_index,
                                            /* [out] */ gvfg_audio_frame_t *out_frame,
                                            /* [in] */ uint32_t timeout_ms);
gvfg_status_t gvfg_release_channel_audio_frame(/* [in] */ gvfg_handle handle,
                                               /* [in] */ int channel_index,
                                               /* [in] */ const gvfg_audio_frame_t *frame);
```

- 預設只擷取 video。CH0 可在 start 前用 `gvfg_set_channel_audio_enabled()` 啟用 audio。
- `gvfg_start_channel()` 會依 audio enabled 狀態開始 video-only 或 video + audio capture。
- `gvfg_read_channel_audio_frame()` 取得下一個 PCM frame 並交付借用的 descriptor。使用完成後必須呼叫 `gvfg_release_channel_audio_frame()`。
- 未 release 前再次 read 會回傳 `GVFG_ESTATE`。
- `out_frame` 為 NULL 或 release token 被修改時回傳 `GVFG_EINVAL`。
- `timeout_ms` 遵循其他 read API：`0` 不等待，`GVFG_TIMEOUT_INFINITE` 無限等待，超時回傳 `GVFG_ETIMEOUT`。
- Video 與 audio 應由不同 worker thread read；同一 channel 不可同時執行兩個 audio read。
- Stop/close 後 descriptor 立即失效；要跨越 release/stop 保存 PCM 必須先複製。

2.9 `gvfg_start_channel`

```
gvfg_status_t gvfg_start_channel(/* [in] */ gvfg_handle handle,
                                 /* [in] */ int channel_index);
```

- `handle`：已成功 open 的 session handle。
- `GVFG_OK`：開始擷取，或 channel 已經 running。
- `GVFG_EINVAL`：handle 為 NULL。
- `GVFG_ESTATE`：尚未 open 裝置。

- `GVFG_ETIMEOUT`：本次 Start 的即時 signal query 顯示沒有 lock；channel 不會進入 running。
- `GVFG_EBUSY`：裝置或 capture resource 正在使用中。
- `GVFG_EIO`：capture 啟動或 I/O 失敗。

Channel 從 stopped 開始新的 running session 時會執行一次新的 signal query，不沿用 停止前的 signal cache。對已 running 的 channel 再次呼叫 Start 會直接回傳 `GVFG_OK`，不會重新查詢訊號。無訊號時 Application 應等待輸入恢復後再次呼叫 Start。

2.10 `gvfg_read_channel_frame`

```
gvfg_status_t gvfg_read_channel_frame(/* [in] */ gvfg_handle handle,
                                      /* [in] */ int channel_index,
                                      /* [out] */ gvfg_frame_t *out_frame,
                                      /* [in] */ uint32_t timeout_ms);
```

- `handle`：running session。
- `out_frame`：接收 frame descriptor，不可為 NULL；呼叫前建議清零。
- `timeout_ms`：0 為 non-blocking；`GVFG_TIMEOUT_INFINITE` 為無限等待；其他值為毫秒。
- `GVFG_OK`：成功取得 frame，caller 現在持有一個必須 release 的 token。
- `GVFG_EINVAL`：handle 或 output pointer 為 NULL。
- `GVFG_ESTATE`：未 running、已有 held frame、已有另一個 read，或等待時被 stop。
- `GVFG_ETIMEOUT`：期限內沒有 frame。
- `GVFG_EBUSY` 表示 capture resource 正在使用中；`GVFG_EIO` 表示 capture 或 I/O 失敗；`GVFG_ENOTSUP` 表示 SDK 不支援該格式。

Copy mode 與 zero-copy mode 都會回傳借用的 frame buffer。兩者的 pointer 都只保證 有效到對應的 `gvfg_release_channel_frame()`。

2.11 `gvfg_release_channel_frame`

```
gvfg_status_t gvfg_release_channel_frame(/* [in] */ gvfg_handle handle,
                                         /* [in] */ int channel_index,
                                         /* [in] */ const gvfg_frame_t *frame);
```

- `handle`：取得該 frame 的同一個 handle。
- `frame`：`gvfg_read_channel_frame()` 原封不動回傳的完整 descriptor。
- `GVFG_OK`：成功歸還 frame。
- 即使輸入訊號在持有 frame 期間中斷，application 仍應對原本成功取得的 frame 呼叫一次 release。
- `GVFG_EINVAL`：NULL 或 token 內容與 held frame 不符。
- `GVFG_ESTATE`：channel 未開啟，或目前沒有可 release 的 held frame。

2.12 `gvfg_gpu_convert_to_buffer`

```
gvfg_status_t gvfg_gpu_convert_to_buffer(
    /* [in] */ const gvfg_frame_t *source,
    /* [in] */ const gvfg_gpu_output_buffer_t *output);
```

- `source`：有效的 YUY2/Y210 frame；若來自 read，必須尚未 release。
- `output`：caller 填好的 destination descriptor。
- `GVFG_OK`：同步轉換及 copy 完成。
- `GVFG_EINVAL`：NULL、尺寸、stride、buffer size、奇偶尺寸或 overflow 錯誤。
- `GVFG_ENOTSUP`：input/output format 不支援。
- `GVFG_EIO`：D3D/GPU resource、dispatch 或 readback 失敗。

2.13 `gvfg_gpu_convert_to_bgra8`

```
gvfg_status_t gvfg_gpu_convert_to_bgra8(
    /* [in] */ const gvfg_frame_t *source,
    /* [out] */ void *destination,
    /* [in] */ uint64_t destination_size,
    /* [in] */ int row_bytes);
```

- `source`：有效且尚未 release 的 YUY2/Y210 frame。
- `destination`：caller-owned BGRA8 buffer。
- `destination_size`：buffer byte 數，至少 `row_bytes * source->height`。
- `row_bytes`：destination stride，至少 `source->width * 4`。
- 回傳狀態與通用 GPU conversion 相同。

2.14 `gvfg_gpu_convert_to_rgb10a2`

```
gvfg_status_t gvfg_gpu_convert_to_rgb10a2(
    /* [in] */ const gvfg_frame_t *source,
    /* [out] */ void *destination,
    /* [in] */ uint64_t destination_size,
    /* [in] */ int row_bytes);
```

參數與 BGRA8 wrapper 相同，但 destination layout 是 packed RGB10A2；每列仍至少 `source->width * 4` bytes。回傳狀態與通用 GPU conversion 相同。

2.15 `gvfg_gpu_convert_to_nv12`

```
gvfg_status_t gvfg_gpu_convert_to_nv12(
    /* [in] */ const gvfg_frame_t *source,
    /* [out] */ void *destination,
    /* [in] */ uint64_t destination_size,
    /* [in] */ int row_bytes);
```

- `source`：有效且尚未 release 的 YUY2/Y210 frame；width、height 必須為偶數。
- `destination`：caller-owned NV12 buffer。
- `destination_size`：至少 `row_bytes * (height + height / 2)`。
- `row_bytes`：Y 與 UV plane 共用的 stride，至少為 `width`。
- 回傳狀態與通用 GPU conversion 相同。

2.16 `gvfg_poll_channel_event`

```
gvfg_status_t gvfg_poll_channel_event(/* [in] */ gvfg_handle handle,
                                      /* [in] */ int channel_index,
                                      /* [out] */ gvfg_event_type_t *out_event_type,
                                      /* [in] */ uint32_t timeout_ms);
```

- `handle`：running session。
- `out_event_type`：接收一個 `gvfg_event_type_t` 事件種類，不可為 NULL。
- `timeout_ms`：規則與 read 相同。
- `GVFG_OK`：成功取出一個事件。
- `GVFG_EINVAL`：handle 或 output pointer 為 NULL。
- `GVFG_ESTATE`：未 running，或等待時被 stop。
- `GVFG_ETIMEOUT`：期限內沒有事件。

2.17 `gvfg_stop`

```
gvfg_status_t gvfg_stop(/* [in] */ gvfg_handle handle);
```

- `handle`：session handle，不可為 NULL。
- `GVFG_OK`：成功；已經 stopped 也視為成功。
- `GVFG_EINVAL`：handle 為 NULL。

此函式會停止 capture、喚醒等待中的 read/poll，並使未 release frame 失效。

2.18 `gvfg_stop_channel`

```
gvfg_status_t gvfg_stop_channel(/* [in] */ gvfg_handle handle,
                                /* [in] */ int channel_index);
```

- 只停止指定 channel；`gvfg_stop()` 會停止同一 handle 已開啟的所有 channel。
- 尚未 open 的 channel 回傳 `GVFG_ESTATE`。

2.19 `gvfg_get_channel_signal_status`

```
gvfg_status_t gvfg_get_channel_signal_status(
    /* [in] */ gvfg_handle handle,
    /* [in] */ int channel_index,
    /* [out] */ gvfg_signal_status_t *out_status);
```

- `handle`：已 open 的 session。
- `out_status`：接收 signal status，不可為 NULL。
- `GVFG_OK`：查詢成功；沒有訊號時仍成功，但 `connected == 0`。
- `GVFG_EINVAL`：handle 或 output pointer 為 NULL。
- `GVFG_ESTATE`：尚未 open 裝置。

- Capture I/O 失敗時可能回傳 `GVFG_EIO`。

Channel 尚未 running 時，此 API 會更新目前的 signal status；running 期間回傳最近一次 觀察到的狀態。Format changed、input plug-in 與 input unplug event 發生後，後續查詢會 反映更新後的狀態。

2.20 `gvfg_get_channel_runtime_info`

```
gvfg_status_t gvfg_get_channel_runtime_info(
    /* [in] */ gvfg_handle handle,
    /* [in] */ int channel_index,
    /* [out] */ gvfg_runtime_info_t *out_info);
```

- `handle`：已 open 或 running 的 session。
- `out_info`：接收 runtime statistics，不可為 NULL。
- `GVFG_OK`：查詢成功。
- `GVFG_EINVAL`：handle 或 output pointer 為 NULL。
- `GVFG_ESTATE`：指定 channel 尚未 open。

2.21 `gvfg_get_version`

```
const char *gvfg_get_version(void);
```

- 回傳目前實際載入的 GVFG runtime DLL 版本，例如 `"2.1.1"`。
- 回傳值是靜態 null-terminated 字串，caller 不可 free。
- 可用於 log、問題回報，以及確認 header、DLL 是否來自同一版本。

2.22 `gvfg_pixel_format_name`

```
const char *gvfg_pixel_format_name(/* [in] */ int pixel_format);
```

- `pixel_format`：`gvfg_pixel_format_t` 或其他整數值。
- 回傳：靜態英文字串 `"YUY2"`、`"Y210"` 或 `"UNKNOWN"`。Caller 不可 free。

2.23 `gvfg_strerror`

```
const char *gvfg_strerror(/* [in] */ gvfg_status_t status);
```

- `status`：任一 GVFG status code。
- 回傳：靜態、null-terminated 英文說明字串；未知值回傳 unknown 類型說明。 Caller 不可 free。

2.24 `gvfg_get_channel_last_sdk_error_detail`

```
gvfg_status_t gvfg_get_channel_last_sdk_error_detail(/* [in] */ gvfg_handle handle,
                                                     /* [in] */ int channel_index,
                                                     /* [out] */ char *out_message,
                                                     /* [in] */ uint32_t out_message_size);
```

此函式提供公開 `GVFG_E*` 失敗狀態的補充說明。回傳內容只描述應用程式可採取行動的 公開錯誤資訊。

- 複製指定 channel 最近一次 fault 或被拒絕操作的 UTF-8 詳細說明；即使該 channel open 失敗仍可查詢。
- `gvfg_read_channel_frame()` / `gvfg_poll_channel_event()` 的 timeout、non-blocking 無資料， 以及正常 stop 喚醒 waiter 都不會覆寫此內容。
- 每次呼叫 `gvfg_start_channel()` 時會清除先前保存的錯誤說明。
- 同一 channel 若被多執行緒同時操作，內容可能被後續錯誤覆蓋；應在失敗後立即取得。
- `GVFG_EINVAL` ：handle/message 為 NULL、channel index 無效，或 buffer size 為 0。

2.25 `gvfg_measure_osd_text`

```
gvfg_status_t gvfg_measure_osd_text(
    /* [in] */ const char *text,
    /* [in] */ uint32_t text_length,
    /* [in] */ uint32_t font_size_px,
    /* [out] */ uint32_t *out_width,
    /* [out] */ uint32_t *out_height);
```

- 不需要 capture handle，不會開啟 device，可在 Start 前量測。
- `text` ：1–32 個可列印 ASCII 字元（ `0x20` – `0x7e` ），不接受換行。
- `font_size_px` ：必須大於 0；字型、單行排版與 SDK OSD 繪製相同。
- `out_width` 、 `out_height` ：不可為 NULL，接收單位為 pixel 的保守完整文字框。 包含排版空白、glyph overhang 及四邊各 1 px 安全邊，不是 pixel-tight 字形邊界。
- `GVFG_OK` ：量測成功。
- `GVFG_EINVAL` ：pointer、文字、長度或字號無效，或量測結果超過可表示範圍。
- `GVFG_EIO` ：文字量測或配置失敗。失敗時可寫入的 output 會歸零。

2.26 `gvfg_set_channel_osd` / `gvfg_clear_channel_osd`

```
gvfg_status_t gvfg_set_channel_osd(
    /* [in] */ gvfg_handle handle,
    /* [in] */ int channel_index,
    /* [in] */ const gvfg_channel_osd_config_t *config);
gvfg_status_t gvfg_clear_channel_osd(
    /* [in] */ gvfg_handle handle,
    /* [in] */ int channel_index);
```

- Channel 必須已 open；可在 Start 前或擷取期間設定或停用。
- `set` 同步複製設定，套用於後續 YUY2 / Y210 frame；文字顏色為 BT.709 limited range， 由 GPU 混合。

- `x/y` 是量測文字框左上角。SDK 依每個 frame 把位置限制在 `[0, frame.width - text_width]` / `[0, frame.height - text_height]`。
- 文字框本身比 frame 大時暫不疊字，擷取仍繼續；設定、frame 尺寸或格式變更後重新判斷。 可容納時恢復 OSD，不縮字、截字或換行。
- `clear` 停用後續 frame 的 OSD。已取得的 frame 仍照原本生命週期使用與 release。
- `GVFG_OK`：設定或停用成功；不表示文字已完成繪製或一定能放入當前 frame。
- `GVFG_EINVAL`：NULL handle/config、無效文字長度、字元或零字號。
- `GVFG_ESTATE`：指定 channel 尚未 open；其他處理失敗可回傳 `GVFG_EIO`。
- 啟用 OSD 的 frame 可能為 SDK-owned output，即使擷取使用 zero-copy； 每次成功 read 仍須恰好 release 一次。

3. Playback 使用的影像型別

以下型別定義於 `gvfg_preview.h`，由 `gvfg_playback.h` 引入。 此標頭只提供 Playback 使用的影像與統計型別；播放操作見第 4 節。

3.1 Preview 型別

本節說明 Playback 使用的資料型別；操作函式與錯誤碼見第 4 節。

`gvfg_preview_pixel_format_t`：

成員	值	說明
`GVFG_PREVIEW_PIXFMT_YUY2`	1	Packed 8-bit YUV 4:2:2，Y0 U0 Y1 V0；Capture 格式應明確對應
`GVFG_PREVIEW_PIXFMT_Y210`	2	Packed 10-bit YUV 4:2:2，16-bit containers
`GVFG_PREVIEW_PIXFMT_GRAY16`	4	每 pixel 一個 16-bit unsigned grayscale component
`GVFG_PREVIEW_PIXFMT_RGBA16`	5	每 pixel 四個 16-bit UNORM component：R、G、B、A
`GVFG_PREVIEW_PIXFMT_DICOM_YBR_FULL_422`	6	每 pixel pair 四個 16-bit words：Y0、Y1、Cb、Cr，數值依 bit_depth 靠右對齊

`gvfg_preview_frame_t`：

欄位	說明
`data`、`data_size`	Caller-owned frame pointer 與 byte 數
`width`、`height`	Frame pixel 尺寸
`pixel_format`	`gvfg_preview_pixel_format_t`
`bit_depth`	Frame bit depth
`row_bytes`	Source row stride，通常對應 `gvfg_frame_t.row_stride_bytes`
`frame_id`	Frame 識別值，通常對應 `gvfg_frame_t.frame_id`

`gvfg_preview_info_t`：

欄位	說明
`active`	非 0 表示 preview pipeline 已啟用
`width` 、 `height` 、 `bit_depth`	最近使用的 frame 資訊
`pixel_format[32]`	Null-terminated 格式名稱
`adapter_name[160]`	Null-terminated D3D adapter 名稱
`adapter_index`	使用的 adapter 索引

`gvfg_preview_stats_t` ：

欄位	說明
`present_fps`	最近五秒成功 Present 的速率
`presented_frames`	成功 Present 的累積 frame 數
`skipped_presents`	底層 Preview 的 Present busy、submit busy 與 queue age 等略過累積；不含所有 core 提前退休

`gvfg_preview_delivery_stats_t` ：以下 counter 從 create 或前一次 shutdown 開始累積； configure 與 clear 不會重設。 `presented` 表示 DXGI 接受 Present，不代表已實際 scanout。

欄位	說明
`submitted`	Preview 接收到並納入 delivery 統計的 render request 數
`presented`	已被 DXGI Present 接受的 frame 數
`replaced`	在 50 ms age limit 後被捨棄的 queued frame 數
`busy`	遇到 upload slot busy 或 Present busy 的 request 數
`failed`	已提交但後續處理失敗的 frame 數
`cancelled`	Shutdown 等流程取消的 frame 數
`in_flight`	目前仍在 pipeline 中的 frame 數
`last_presented_id`	最近成功 Present 的 `frame_id`
`present_busy`	因 Present busy 而略過的累積次數
`slots_busy`	因內部 upload slot busy 而略過的累積次數
`last_busy_id`	最近一次 busy frame 的 `frame_id`

正常情況下：

```
submitted = presented + replaced + busy + failed + cancelled + in_flight
```

4. 統一 Playback API

公開標頭為 `gvfg_playback.h` ，連結 `gvfg_playback.lib` 、部署 `gvfg_playback.dll` 。 MinGW 使用 `libgvfg_playback.dll.a` 。

`gvfg_playback` 是 opaque handle。`config.struct_size` 、`stats.struct_size` 須設為對應結構大小；config 可先呼叫 `gvfg_playback_default_config()` 填好預設值。 音訊格式 `gvfg_audio_playback_format_t` 包含 sample_rate、channels、bits_per_sample， 目前支援 signed interleaved PCM16。

函式	行為
`gvfg_playback_default_config(config)`	預設影音模式、External master、100 ms preroll、48 kHz stereo PCM16
`gvfg_playback_create(out)`	建立 handle，不啟動輸出
`gvfg_playback_attach_window(p, hwnd)`	設定 HWND；NULL 表示無畫面輸出但仍消耗影像時間軸
`gvfg_playback_start(p, config)`	啟動選定的純影像、純音訊或影音模式
`gvfg_playback_submit_video(p, frame)`	複製影像與 timestamp，排入共用播放佇列
`gvfg_playback_submit_audio(p, pcm, bytes, timestamp_ns)`	複製 PCM，以 SDK 交付時間建立來源 sample 時間軸
`gvfg_playback_present_image(p, image)`	停止狀態、已綁定 HWND 時同步顯示單張影像，不需 start 或 PTS
`gvfg_playback_set_volume(p, gain)`	PCM16 軟體音量，範圍 0.0 到 2.0
`gvfg_playback_set_sync_enabled(p, enabled)`	0 為各自播放且不自動恢復同步；1 為啟用同步，預設 1
`gvfg_playback_reset_timeline(p)`	丟棄待播資料，使用原設定重新建立時間軸
`gvfg_playback_stop(p)`	停止播放並丟棄待播資料，可重複呼叫
`gvfg_playback_destroy(p)`	停止並銷毀，NULL 可安全忽略
`gvfg_playback_get_stats(p, stats)`	共同緩衝、queue、影音進度、誤差及調速資訊
`gvfg_playback_get_preview_info(p, info)`	影像輸出資訊
`gvfg_playback_get_preview_stats(p, stats)`	GPU / 呈現統計
`gvfg_playback_set_signal_connected(p, connected)`	UI 執行緒轉交訊號狀態：0 顯示 helper 內建 NO SIGNAL 圖卡，1 隱藏
`gvfg_playback_get_delivery_stats(p, stats)`	影像呈現與略過等計數
`gvfg_playback_strerror(status)`	靜態英文錯誤字串，不可 free

狀態碼為 OK、EINVAL、ESTATE、EOUTPUT、EQUEUE_FULL、EDISCONTINUITY、EAUDIO； queue 壓力會自動降級並略過過期資料，不要求 caller 因此停止擷取。正常同步時 External / Audio master 會退休過期預覽影格，並累計 `video_dropped` ；GPU busy 也計為略過，真正 GPU 失敗才回報 EOUTPUT。音訊及獨立錄影不受此預覽策略影響。 2.0.2 的 get_stats 也接受舊 stats 大小，只寫入呼叫端提供的原始結構範圍。 submit 返回後 caller 可立即釋放 SDK frame；present_image 則須保留資料到函式返回。 控制操作（含 present_image）須由 caller 序列化；可由獨立影音 producer 併行 submit。 stop/reset 是取消，不是無損 EOS drain。

完整使用流程與可編譯的提交範例見 統一影音播放使用手冊。

4.1 `gvfg_playback_config_t`

先呼叫 `gvfg_playback_default_config(&config)` ，不需要自行填 struct_size。 start 會複製設定；只接受停止狀態的 handle，不會代替 caller 停掉既有播放。

欄位	預設值	限制與語意
`struct_size`	`sizeof(gvfg_playback_config_t)`	必須等於目前完整結構大小
`mode`	`GVFG_PLAYBACK_AUDIO_VIDEO`	Video-only=1、Audio-only=2、Audio-video=3
`master`	`GVFG_PLAYBACK_MASTER_EXTERNAL`	External=0、Audio=1、Video=2；純影像強制 External，純音訊強制 Audio
`audio_format.sample_rate`	48000	啟用音訊時為 1～384000 Hz；通過參數檢查不代表 Windows 裝置一定支援
`audio_format.channels`	2	啟用音訊時為 1～32
`audio_format.bits_per_sample`	16	啟用音訊時必須為 16，signed interleaved PCM
`preroll_ms`	100	1～1000 ms；啟用串流需要的初始 / 重新緩衝覆蓋範圍
`max_video_frames`	32	1～600，待播影像張數上限；純音訊仍須填合法值
`max_audio_ms`	1000	不小於 preroll_ms，且不超過 10000；包括共用 queue 與輸出端待播量
`max_video_bytes`	256 MiB	必須大於 0；包括目前 worker 處理中的 CPU 影像副本

External / Audio master 的預覽允許過期退休與 GPU-busy 略過，不是逐幀無損輸出。 Video master 不啟用自動過期退休；有限緩衝也不保證在輸出速度不足時永久接收所有 frame。 選擇模式不會改變 Capture SDK 的啟用設定；Capture 的 audio enable 仍由 caller 管理。

4.2 `gvfg_playback_video_frame_t`

欄位	意義
`image`	`gvfg_preview_frame_t`；資料指標、可用 bytes、尺寸、格式、有效 bit depth、row stride、frame ID
`timestamp_ns`	來源共同單調時間軸的交付奈秒值；非 0、各路嚴格遞增、不超過 INT64_MAX

image 支援 YUY2、Y210、GRAY16、RGBA16、DICOM YBR_FULL_422，以 `GVFG_PREVIEW_PIXFMT_*` 指定。Capture 的 `GVFG_PIXFMT_*` 必須明確對應，不能直接複製 enum 整數。 `data_size` 至少覆蓋 `row_bytes * height`；row_bytes 須足夠容納一列指定格式。 submit 會複製實際影像列範圍，frame ID 僅作身分 / 統計資訊，不用來計算 PTS。 影像各欄位與格式 layout 詳見第 3 節。

4.3 `gvfg_playback_submit_audio()` 的輸入

參數	契約
`p`	已 start 且啟用音訊的 handle
`pcm`	非 NULL 的 signed interleaved PCM16，返回前保持有效
`bytes`	大於 0，對齊 channels × 2，每包最多 100 ms
`timestamp_ns`	同一來源時鐘域的 SDK delivery time；須大於包長、非 0、嚴格遞增且不超過 INT64_MAX

第一包來源起點估為 timestamp 減去包長；後續維持連續的 sample 時間軸， 平滑追蹤與交付時鐘的長期偏差。相鄰 delivery timestamp 的間隔若超過 250 ms 加上本包包長， 會進入獨立播放並建立新的音訊映射；不會因此停止擷取。重複或倒退的時間仍會回報 EDISCONTINUITY。 這些規則不會提供硬體擷取時間、漏 sample 數或絕對聲學延遲。

4.4 狀態碼

值	名稱	caller 處理
0	`GVFG_PLAYBACK_OK`	操作成功；submit 成功表示接受，不表示已呈現
-1	`GVFG_PLAYBACK_EINVAL`	修正 NULL、格式、尺寸、對齊、struct_size 或 config 範圍
-2	`GVFG_PLAYBACK_ESTATE`	檢查是否 start、模式是否啟用該路、單張操作是否在停止且已綁窗狀態
-3	`GVFG_PLAYBACK_EOUTPUT`	檢查 WASAPI / GPU / 視窗 / 記憶體及重啟流程；不能假設會自動恢復
-4	`GVFG_PLAYBACK_EQUEUE_FULL`	保留的未接受狀態；先 release SDK frame，不可持有擷取資源等待
-5	`GVFG_PLAYBACK_EDISCONTINUITY`	協調 producer，重設時間軸；新音訊格式須以新 config stop/start
-6	`GVFG_PLAYBACK_EAUDIO`	影音模式的監聽音效已因輸出故障停用，影像繼續；提示一次，繼續獨立擷取 / 錄影。此 handle 不再接受音訊，重建 handle 才重試音效

影音模式的音訊輸出故障不設定整體 `last_error`；音訊 submit 回傳 `EAUDIO`， 影像 submit 繼續。影音 start 遇到音效開啟失敗仍可成功，音訊 submit 會回報 `EAUDIO`。 停用狀態跨 stop/start/reset 保留；重建 handle 才重試音效。純音訊模式仍視為輸出失敗。 其他非同步輸出錯誤會保存在 `last_error`，後續 submit 會回傳此錯誤。 `running` 仍可能為 1，因此不能只看 running 判斷輸出健康。

4.5 `gvfg_playback_stats_t`

一般 Customer Demo 不需要定期顯示本表；診斷或整合工具可自行查詢。 先設 `stats.struct_size = sizeof(stats)`。getter 也接受 2.0.0 / 2.0.1 與 2.0.2 / 2.0.3 的 stats 大小， 只回填舊結構範圍；config 並未因此接受任意大小。

欄位	定義
`struct_size`	caller 的有效結構大小
`running`	Playback 是否已 start 且尚未 stop，不是輸出健康保證
`buffering`	正在等初始 / 重新緩衝，或因持續錯誤暫停
`last_error`	共用 worker 的持續錯誤；查詢 API 返回 OK 不表示此欄也為 OK
`video_submitted`	成功接受的來源影像張數
`video_processed`	已呈現、隱藏消化或已計數略過的張數
`video_presented`	成功被 DXGI Present 接受的張數，不代表完成螢幕 scanout
`audio_submitted`	成功接受的音訊 packet 數，不是 sample 數
`queue_rejections`	保留的容量拒收計數；一般 queue 壓力會自動降級 / 略過，不增加此值
`rebuffer_count`	已運行時間軸重新進入共同緩衝的次數

欄位	定義
`video_queue_bytes`	CPU 影像副本 bytes，包含 worker 目前處理中的副本
`video_queue_frames`	共用待播 deque 張數，不含已移交 worker 的單張
`audio_queue_ms`	共用 PCM queue 加上音訊輸出端待播 frames 的估計毫秒數
`video_timestamp_ns`	最近完成影像處理所記錄的來源時間；隱藏模式也可推進，不是實體 scanout 時間
`audio_timestamp_ns`	已消耗音訊 frames 換算的來源位置估計，不是麥克風量得的發聲時間
`av_difference_ns`	worker 保存的 audio 減 video clock 差值；兩個 clock 有效時才有意義，不是同時刻外部量測
`audio_rate_ratio`	最近給輸出端的速率比例，1.0 表示不補償
`video_dropped`	2.0.2 新增，過期退休與 GPU-busy 等預覽略過數，不是 capture / recording loss
`sync_degraded`	1 表示目前獨立播放，不等待另一路完成同步；0 表示正常同步流程
`sync_fallbacks`	本次 start 後進入獨立播放的次數
`audio_dropped_frames`	從 Playback 共用 queue 丟棄的 PCM sample frames；不含 WASAPI reset 取消的 padding，不是擷取遺失數

不要把 submitted、processed、presented 視為必定相等。初始 / 共同緩衝、隱藏顯示、 過期略過、in-flight 與 stop/reset 取消，都會影響計數關係。

4.6 控制 API 的細節

`gvfg_playback_set_sync_enabled(p, enabled)` 接受 0 或 1，其他值 / NULL handle 回傳 EINVAL。 可在 start 前或運行中呼叫；0 立即使用獨立播放且不自動恢復同步，1 在條件穩定後恢復同步。 設定保留到 stop/start、reset_timeline；重新 create 的 handle 預設為 1。 主動關閉同步也會使 `sync_degraded` 為 1，但不增加 `sync_fallbacks`。

API	狀態、ownership 與限制
`default_config(config)`	NULL 時不操作；其餘填完整預設值與 struct_size
`create(out)`	out 不可 NULL；成功後 caller 擁有 handle，失敗時 out 為 NULL
`attach_window(p, hwnd)`	綁定 HWND；NULL 隱藏輸出但保留模式與影像時間軸；不自動 start
`start(p, config)`	必須為停止狀態；重設計數、queue、時鐘，沿用已設定音量與視窗
`submit_video(p, frame)`	已 start 且啟用影像；返回後無論成功與否，都必須結束 caller 自己的 Capture frame ownership
`submit_audio(p, pcm, bytes, ts)`	已 start 且啟用音訊；成功返回前完成複製
`present_image(p, image)`	停止狀態且有 HWND；同步等待工作完成，資料須有效到返回；GPU busy 不等於實體顯示成功
`set_volume(p, gain)`	finite 0.0 ~ 2.0；可先設定再 start，0 是靜音而不是停音訊

API	狀態、ownership 與限制
`reset_timeline(p)`	stop、清畫面、若原本 active 則用原 config start；未建立呈現表面（swapchain）的空畫面視為清除成功；會取消舊 queue，真正的清畫面或重啟失敗仍回報錯誤
`stop(p)`	可重複呼叫；join 共用 worker、銷毀音訊輸出並清 queue；保留 Preview 物件供後續重用
`destroy(p)`	NULL 安全；stop 後銷毀 Preview 與 handle
`get_stats(p, stats)`	驗證 struct_size，回填共同快照；沒有音訊輸出時不建立它
`get_preview_info(p, info)`	回傳影像資訊；尚未建立 Preview 時為零值
`get_preview_stats(p, stats)`	回傳呈現 FPS / 計數；尚未建立 Preview 時為零值
`set_signal_connected(p, connected)`	僅接受 0 / 1，預設 1。0 顯示 helper 內建圖卡、1 隱藏。可在 attach 前設定，跨 stop/start/reset/reattach 保留；與 attach/destroy 在 HWND 執行緒序列化。圖卡隨視窗縮放，detach/destroy 釋放。此 API 不查詢 Capture、不更動影格或同步、不依 FPS 判斷訊號
`get_delivery_stats(p, stats)`	回傳底層 Preview delivery 計數；不包含共用核心提前退休的所有過期影格
`strerror(status)`	靜態字串，不可 free；未知值回傳 unknown error 文字

函式完整名稱皆為 `gvfg_playback_` 加上表中名稱。getters 的輸出指標不可 NULL。 控制操作含 present_image，須由 caller 序列化；一個影像與一個音訊 producer 可併行 submit。 Playback 不控制 Capture handle，亦不接管它的 read/release 或 channel lifecycle。